STAT 6900: Stochastic Processes in Modern Machine Learning
Lecture 5: Foundations of Reinforcement Learning

Farzad Sabzikar

Iowa State University

March 17, 2026

Sequential Decision Making Under Uncertainty

Many real-world systems evolve dynamically over time.

Key features:

- The current state influences future evolution,
- Decisions affect future probability distributions,
- Performance is evaluated over long horizons.

Contrast with static optimization:

- Static: choose a single decision x to optimize $f(x)$,
- Sequential: choose a sequence $(a_0, a_1, \dots)$ interacting with randomness.

Conclusion. Reinforcement learning studies *sequential decision making under uncertainty*.

Basic Interaction Structure

Reinforcement learning studies an **agent** interacting with an **environment** over time.

At each time step t:

- The system is in a **state** X_t,
- The agent selects an **action** A_t,
- The environment produces:
  - a **reward** R_{t+1},
  - a new state X_{t+1}.

Key feature:

$$\text{The distribution of } X_{t+1} \text{ depends on } (X_t, A_t).$$

Goal: choose actions to optimize long-term performance.

Stochastic Evolution

The system evolves randomly over time.

State dynamics:

$$X_{t+1} \sim P(\cdot \mid X_t, A_t).$$

Interpretation:

- Even with fixed (X_t, A_t), outcomes are random,
- The model is governed by a **transition kernel** P,
- This is a discrete-time stochastic process controlled by actions.

Connection:

- Without control $\rightarrow$ Markov process,
- With control $\rightarrow$ controlled Markov process.

Sources of Uncertainty

Uncertainty arises from multiple sources:

- **Random dynamics:** intrinsic noise in state transitions,
- **Partial observation:** the true state may not be fully observable,
- **Interaction:** presence of other agents or environments,
- **Unknown model:** transition and reward mechanisms must be learned.

Implication: Decisions must be made under both *randomness* and *model uncertainty*.

What Are We Optimizing?

Performance is evaluated over time via cumulative rewards.

Typical objective:

$$\mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t R_{t+1}\right],$$

where $0 < \gamma < 1$ is a discount factor.

Interpretation:

- Rewards in the near future are weighted more heavily,
- The objective balances short-term and long-term gains.

Key idea: We optimize a *functional of the entire trajectory*, not a single decision.

Policies

A **policy** describes how the agent selects actions.

- Deterministic policy:
$$A_t = \pi(X_t),$$
- Stochastic policy:
$$A_t \sim \pi(\cdot \mid X_t).$$

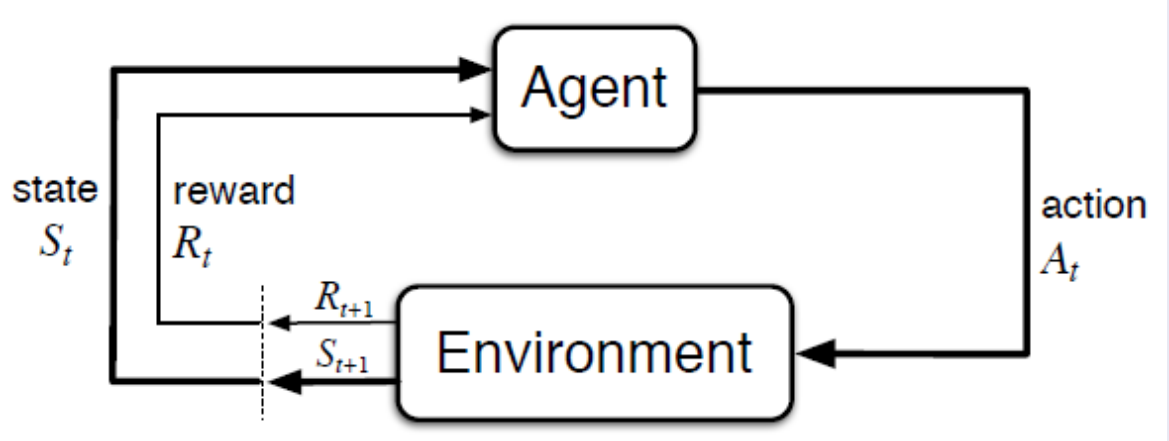
Goal: Find a policy π that maximizes expected cumulative reward.

Key shift:

Optimize over functions π, not single decisions.

Robot navigation / control (Sutton–Barto)

Example 5.1



Setup.

- State: X_t = robot location,
- Action: A_t (move in grid),
- Reward: R_{t+1} depends on position.

Dynamics.

$$X_{t+1} \sim P(\cdot \mid X_t, A_t).$$

Goal. Navigate to high-reward regions and avoid penalties.

Example 5.1

Key difficulty:

- Immediate rewards can be misleading,
- Some actions reduce short-term reward but improve long-term outcome,
- Effects of actions propagate through future states.

Sequential dependence:

$$(X_t, A_t) \longrightarrow X_{t+1} \longrightarrow X_{t+2} \longrightarrow \cdots$$

Implication: Optimal decisions must account for future consequences.

Example 5.1

Objective:

$$\max_{\pi} \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t R_{t+1}\right].$$

Policy:

$$A_t \sim \pi(\cdot \mid X_t).$$

Key takeaway:

- The problem is not a single decision,
- It is the selection of an optimal **policy**,
- This leads to value functions and Bellman equations.

From Example to Mathematical Model

The robot navigation example illustrates a general structure:

- A **state** describing the current situation,
- A set of **actions** available to the agent,
- **Random transitions** depending on state and action,
- **Rewards** measuring performance over time,
- A **policy** governing decisions.

Key idea: All such problems can be formalized using a single mathematical framework.

This framework is called a Markov Decision Process (MDP).

Notation switch: from stochastic-process language to RL language

So far, we used stochastic-process notation such as X_t for the state.

From this point on, we switch to the standard reinforcement-learning notation:

$$X_t \quad \rightsquigarrow \quad S_t, \qquad A_t \text{ stays as the action,} \qquad R_{t+1} \text{ is the reward.}$$

- This matches the notation used in Sutton & Barto.
- Mathematically, nothing changes: we are still studying a controlled stochastic process.

Next: Finite Markov Decision Processes (MDPs)

- The robot example fits a general mathematical structure.
- Goal: formalize the **discrete-time** objects that will later have **continuous-time** analogues.
- We isolate the minimal structure: **state**, **action**, **reward**, **dynamics**, **policy**, **return**, **value**.

Agent–Environment interface

Definition 5.1

Interaction occurs at discrete times $t = 0, 1, 2, \ldots$:

- At time t, the agent observes a **state** $S_t \in \mathcal{S}$.
- The agent selects an **action** $A_t \in \mathcal{A}(S_t)$.
- The environment produces a **reward** $R_{t+1} \in \mathcal{R}$ and a new state S_{t+1}.

Stochastic evolution:

$$(S_{t+1}, R_{t+1}) \sim p(\cdot, \cdot \mid S_t, A_t).$$

A trajectory begins as

$$S_0, A_0, R_1, S_1, A_1, R_2, \ldots \tag{1}$$

Dynamics: the core object $p(s', r \mid s, a)$

Definition 5.2

The evolution described previously is encoded by a probability kernel.

$$p(s', r \mid s, a) \;:=\; \Pr\{S_t = s',\, R_t = r \mid S_{t-1} = s,\, A_{t-1} = a\}. \tag{2}$$

- This *four-argument* p completely characterizes the dynamics.
- S_t and R_t are **jointly determined**.

Markov property

Definition 5.3

The **Markov property** means: given the current state and action, the future is independent of the past.

Formally,

$$\begin{aligned}\Pr\{S_t = s', R_t = r \mid S_{t-1} = s, A_{t-1} = a, \text{past}\} = \\ \Pr\{S_t = s', R_t = r \mid S_{t-1} = s, A_{t-1} = a\}.\end{aligned} \tag{3}$$

From $p(s', r \mid s, a)$ we can derive simpler quantities that are often used in practice. **State-transition probabilities**

$$p(s' \mid s, a) \;:=\; \Pr\{S_{t+1} = s' \mid S_t = s, A_t = a\} = \sum_{r \in \mathcal{R}} p(s', r \mid s, a). \tag{4}$$

Transition probabilities and expected rewards

Definition 5.4

Expected rewards for state–action pairs

$$r(s,a) \;:=\; \mathbb{E}[R_t \mid S_{t-1}=s, A_{t-1}=a] = \sum_{r\in\mathcal{R}} r \sum_{s'\in\mathcal{S}} p(s', r \mid s, a). \tag{5}$$

Expected rewards conditioned on next state

$$r(s,a,s') \;:=\; \mathbb{E}[R_t \mid S_{t-1}=s, A_{t-1}=a, S_t=s'] = \sum_{r\in\mathcal{R}} r \, \frac{p(s', r \mid s, a)}{p(s' \mid s, a)}. \tag{6}$$

- $r(s,a)$ averages over all possible next states and rewards.
- $r(s,a,s')$ gives the expected reward when the next state is fixed to be s'.

Return (unified notation)

Definition 5.5

Let T denote the terminal time of an episode (possibly $T = \infty$).

The **return** from time t is

$$G_t := \sum_{k=t+1}^{T} \gamma^{k-t-1} R_k, \tag{7}$$

allowing either $T = \infty$ or $\gamma = 1$ (but not both), so that the sum remains well-defined.

A key recursion is

$$G_t = R_{t+1} + \gamma G_{t+1}, \tag{8}$$

with the convention $G_T = 0$ for terminal time T.

- G_t is the total future reward seen from time t.
- This recursion is the basic reason Bellman equations work.

From return to recursion

Recall the return satisfies the recursion

$$G_t = R_{t+1} + \gamma G_{t+1}. \tag{9}$$

This shows that long-term performance can be characterized **recursively**.

- Instead of working with the full sum G_t, we can study a function that satisfies a **fixed-point equation**.
- This leads to the notion of a **value function** and the **Bellman equation**.

Key idea:

recursion in discrete time $\longrightarrow$ differential equation in continuous time (HJB).

Policies

Definition 5.6

A **policy** π is a mapping from states to probability distributions over actions.

If the agent follows π, then $\pi(a \mid s)$ denotes the probability that

$$A_t = a \quad \text{given} \quad S_t = s. \tag{10}$$

- A policy specifies *how decisions are made* at each state.
- In reinforcement learning, the objective is to optimize over policies rather than over single actions.

State-value function v^π

Definition 5.7

The **state-value function** under policy π is the expected return starting from state s and following π thereafter:

$$v^\pi(s) := \mathbb{E}_\pi[G_t \mid S_t = s] = \mathbb{E}_\pi\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \,\middle|\, S_t = s\right], \qquad s \in \mathcal{S}. \tag{11}$$

- The value of any terminal state (if present) is 0.
- v^π turns a random trajectory objective into a deterministic function of s.
- It answers the question: *how good is it to start from state s under policy π?*

Example (control-flavored): Pole-balancing

Example 5.2

Story (short bullets).

- Apply forces to a cart on a track to keep a hinged pole from falling.
- Failure occurs if the pole angle exceeds a threshold or the cart runs off the track.
- After failure, the pole is reset and the attempt restarts.

Key RL ingredients (informal).

- *State*: cart position/velocity; pole angle/angular velocity.
- *Action*: applied force to the cart.
- *Reward design*: encodes "balance as long as possible".

Why this example is important: it is naturally interpreted as feedback control of a physical system.

Example 5.2

Two standard formulations:

- **Episodic:** reward $+1$ at each time step *without* failure.
  - Then G_t equals the number of steps until failure.
- **Continuing with discounting:** reward -1 on each failure, and 0 otherwise.
  - Then G_t is related to γ^K where K is the number of time steps before failure.

Why this example matters for our goal: it is naturally viewed as control of an underlying physical system, making it a useful prototype for continuous-time models.

Optimal policies and optimal value functions

Definition 5.8

A policy π is better than or equal to π' if

$$v^{\pi}(s) \geq v^{\pi'}(s) \quad \text{for all } s \in \mathcal{S}. \tag{12}$$

There exists at least one **optimal policy** (possibly not unique), denoted π^*.

Optimal state-value function

$$v^*(s) := \max_{\pi} v^{\pi}(s). \tag{13}$$

Optimal action-value function

$$q^*(s, a) := \max_{\pi} q^{\pi}(s, a). \tag{14}$$

Relation between q^* and v^*

Example 5.3

They satisfy the relation

$$q^*(s,a) = \mathbb{E}[R_{t+1} + \gamma v^*(S_{t+1}) \mid S_t = s, A_t = a]\,. \tag{15}$$

- $q^*(s,a)$: value of taking action a first, then acting optimally.
- $v^*(s)$: best achievable long-run performance from state s.

Interpretation:

$$\text{action value} = \text{one-step reward} + \text{optimal continuation value}.$$

Bellman optimality equation for v^*

Proposition 5.1

The optimal state-value function satisfies the **Bellman optimality equation**:

$$v^*(s) = \max_{a \in \mathcal{A}(s)} \sum_{s' \in \mathcal{S}} \sum_{r \in \mathcal{R}} p(s', r \mid s, a) \left[r + \gamma v^*(s') \right]. \tag{16}$$

This is the dynamic programming principle in discrete time: *optimize via one-step look-ahead + continuation value.*

- Local optimization at one step determines global long-run optimality.
- This is the discrete-time analogue of the HJB principle in continuous time.

Bellman optimality equation for q^*

Proposition 5.2

The optimal action-value function satisfies:

$$q^*(s,a) = \sum_{s' \in \mathcal{S}} \sum_{r \in \mathcal{R}} p(s', r \mid s, a) \left[r + \gamma \max_{a' \in \mathcal{A}(s')} q^*(s', a') \right]. \tag{17}$$

This form is central algorithmically: greedy selection uses $\arg\max_{a'} q^*(s', a')$.

It is mathematically equivalent to the state-value formulation, but often more convenient for decision rules.

Gridworld as a concrete finite MDP

Example 5.4

- States are cells of a rectangular grid.
- Actions: north/south/east/west; moves are deterministic.
- Actions that would move off-grid keep the state unchanged and incur a penalty.
- Special states produce positive reward and transition to designated successor states.

Purpose of the example: a fully explicit finite MDP where $p(s', r \mid s, a)$ can be written down exactly.

Why this matters: it shows that Bellman equations are not abstract—they become concrete systems once the model is specified.

A bridge to continuous time

- The **discrete-time** dynamic programming principle has the form

$$\text{(one-step reward)} + \text{(discounted continuation value)}.$$

- The key object is the **value function** together with its **optimality equation**.
- In continuous time, the same principle becomes an *infinitesimal* statement:
  - discrete transition kernel $p(\cdot \mid s, a) \longrightarrow$ drift/diffusion of a controlled process,
  - Bellman optimality equation $\longrightarrow$ HJB equation.
- Thus, the discrete theory we have developed is the direct precursor of controlled diffusions and HJB.

Next: Dynamic Programming

- Goal: develop the **Bellman / Dynamic Programming** viewpoint for solving finite MDPs.
- Dynamic programming assumes a **model** of the environment dynamics, so the relevant expectations can be computed.
- Output: value functions and optimal policies via **Bellman equations** and **Bellman operators**.

Dynamic programming viewpoint

Dynamic programming solves a sequential decision problem by exploiting:

- **optimal substructure:** an optimal strategy has optimal "tails",
- **overlapping subproblems:** many trajectories share the same future states.

In MDPs this is encoded by **Bellman equations**:

- *Policy evaluation*: Bellman equation for a *fixed* policy,
- *Optimal control*: Bellman *optimality* equation (maximum over actions).

Bellman expectation equation for a fixed policy

Proposition 5.3

For a fixed policy π, the value function v^π satisfies the Bellman expectation equation:

$$v^\pi(s) = \sum_a \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma v^\pi(s') \right], \qquad s \in \mathcal{S}. \tag{18}$$

Interpretation: *value = expected immediate reward + discounted continuation value.*

Bellman expectation equation for q^π

Proposition 5.4

For a fixed policy π, the action-value function q^π satisfies:

$$q^\pi(s,a) = \sum_{s',r} p(s',r \mid s,a) \left[r + \gamma \sum_{a'} \pi(a' \mid s')\, q^\pi(s',a') \right]. \tag{19}$$

This makes explicit: after the first action a, future actions are selected using $\pi(\cdot \mid s')$.

Iterative policy evaluation

Definition 5.9

Policy evaluation computes v^π by successive approximation. Starting from any v_0, define a sequence $\{v_k\}$ by repeatedly applying the Bellman expectation backup:

$$v_{k+1}(s) = \sum_a \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma v_k(s') \right], \qquad s \in \mathcal{S}. \tag{20}$$

- This is the **synchronous** form: all states are updated using the previous iterate v_k.
- Variants update states *in place* (asynchronous / Gauss–Seidel style).

Policy improvement: greedy w.r.t. a value function

Definition 5.10

Given a value function v, a **greedy** policy π_{greedy} chooses actions that maximize the one-step lookahead objective:

$$\pi_{\text{greedy}}(s) \in \arg\max_{a} \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma v(s') \right]. \tag{21}$$

In particular, using $v = v^{\pi}$ yields a policy improvement step starting from π.

- This is a local maximization step based on the current value function.
- It is the discrete-time analogue of maximizing the Hamiltonian in continuous time.

Policy improvement theorem

Theorem 5.1

Let π be any policy, and define a new policy π' that is greedy with respect to v^π:

$$\pi'(s) \in \arg\max_a \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma v^\pi(s') \right]. \tag{22}$$

Then π' is at least as good as π:

$$v^{\pi'}(s) \geq v^\pi(s) \qquad \text{for all } s \in \mathcal{S}. \tag{23}$$

This shows that Bellman-based local improvement leads to global performance improvement.

Policy iteration: evaluation + improvement

Definition 5.11

Policy iteration alternates:

- **(Evaluation)** compute v^{π_k} (exactly or approximately) using policy evaluation;
- **(Improvement)** update $\pi_k \mapsto \pi_{k+1}$ greedily with respect to v^{π_k}.

The improvement step can be written:

$$\pi_{k+1}(s) \in \arg\max_a \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma v^{\pi_k}(s') \right]. \tag{24}$$

Thus policy iteration combines Bellman evaluation with repeated policy improvement.

Value iteration

Definition 5.12

Value iteration applies the Bellman *optimality* backup directly. Starting from any v_0, define:

$$v_{k+1}(s) = \max_a \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma v_k(s') \right], \qquad s \in \mathcal{S}. \tag{25}$$

- This is a fixed-point iteration for the optimality equation.
- A greedy policy w.r.t. v_k is typically near-optimal once v_k is close to v^*.

Bellman optimality operator

Definition 5.13

Define the **Bellman optimality operator** T acting on a value function v by

$$(Tv)(s) := \max_a \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma v(s') \right]. \tag{26}$$

Then value iteration is $v_{k+1} = Tv_k$ and the optimal value function satisfies the fixed-point equation

$$v^* = Tv^*. \tag{27}$$

Asynchronous DP (conceptual)

Definition 5.14

DP backups need not update all states in lockstep. **Asynchronous DP** updates some states more often than others:

- update a single state (or a subset) at each step;
- possibly use the most recent values (in-place updates).

The one-state optimality backup has the same form as (25), but applied only to selected states.

Generalized Policy Iteration (GPI)

Definition 5.15

Generalized Policy Iteration is the idea that:

- one process *evaluates* a policy (driving v toward v^π),
- another process *improves* the policy (driving π toward greedy w.r.t. v),

and these two processes can run *simultaneously* rather than in strict alternation.

This viewpoint connects DP methods (model-based backups) to learning methods (data-driven backups) studied later.